

Real-time translation, built like a conversation, not a feature.

How OHUN turns speech and text across languages into a single live conversation — the pipeline, the peer-to-peer calling architecture, and the constraints we chose to name instead of hide.

Published

September 2026

Prepared by

The OHUN team

Stack

Next.js · Supabase · AssemblyAI · Claude · WebRTC

Status

Public preview

ABSTRACT

OHUN is a real-time translation layer for ordinary conversation — chat, voice calls, video calls, and group calls — built so that each person always speaks, types, and hears their own language. This paper describes the system's architecture: a shared transcribe-translate-deliver pipeline built on AssemblyAI and Anthropic Claude, a peer-to-peer WebRTC calling layer that keeps audio and video off our servers entirely, and the specific engineering tradeoffs — including where the product deliberately does *not* support a language yet — that came out of building it against a real, narrow speech-to-text constraint rather than a hypothetical one.

Introduction

Two people who don't share a language have exactly two options today: route the conversation through a human interpreter, or route it through a translation app that turns talking into typing — read a message, type a reply, wait, repeat. Both work. Neither feels like talking to someone.

OHUN's premise is narrow on purpose: keep the shape of a normal conversation — you speak, they respond, in real time — and put the translation *inside* that shape instead of replacing it. That means the transcription, translation, and delivery all have to happen fast enough and reliably enough to disappear, whether the conversation is a typed message, a voice note, a phone call, or a seven-person group call.

System overview

Every surface in OHUN — chat, 1:1 calls, group calls — is a different transport wrapped around the same translation loop. What differs is only *how the source and the result travel*: a chat message is text in and text out, computed once and cached; a call is a live audio stream in, and a spoken, captioned translation out, repeated for as long as the call runs.

SURFACE	INPUT	TRANSCRIPTION	DELIVERY
Chat message	Typed text	—	Translated text, cached per language
Voice note	Recorded audio	AssemblyAI batch	Transcript + translated text
1:1 / group call	Live microphone	AssemblyAI realtime streaming	Spoken aloud + live captions

Table 1 – one translation loop, three transports.

The translation pipeline

The loop itself has three stages, and the same three stages run whether the source is a call, a voice note, or a typed message that happens to need no transcription step at all.

- **Speech-to-text** AssemblyAI

Calls use Universal-Streaming, transcribing turn by turn as you talk. Voice notes use AssemblyAI's batch API instead, which covers a broader set of languages than realtime streaming does — the reason a voice note and a call don't have identical language support (\$6).

- **Translation** CLaude

Finished utterances, not partial fragments, are translated for meaning rather than word-for-word — preserving tone and recovering gracefully from the occasional speech-recognition slip a literal pass would just propagate.

- **Delivery**

Captioned on both sides always; on a call, also spoken aloud with the browser's Web Speech API, while the original speaker's raw voice is briefly ducked so the translation stays intelligible.

Figure 1 — the transcribe → translate → deliver loop.

Translating on completed utterances rather than mid-sentence is a deliberate latency trade: it costs a short pause after each sentence, in exchange for translating with the full sentence as context instead of a fragment that can flip meaning by its end.

Real-time calling architecture

Calls — 1:1 and group — are peer-to-peer WebRTC. Supabase Realtime is used purely as a signalling channel: it carries SDP offers and answers and ICE candidates between participants, and nothing else. Audio and video never pass through OHUN's servers at any point.

A 1:1 call negotiates a single `RTCPeerConnection` using a *perfect negotiation* pattern — each side is deterministically polite or impolite, so a local renegotiation (turning on a camera, say) and an incoming offer arriving at the same instant resolve without a race. Every SDP mutation on a connection is serialized through a promise queue for the same reason.

Group calls: full mesh

A group call is a full mesh of `RTCPeerConnections` — every participant holds one connection to every other participant, with their microphone and any camera they've enabled riding those same connections.

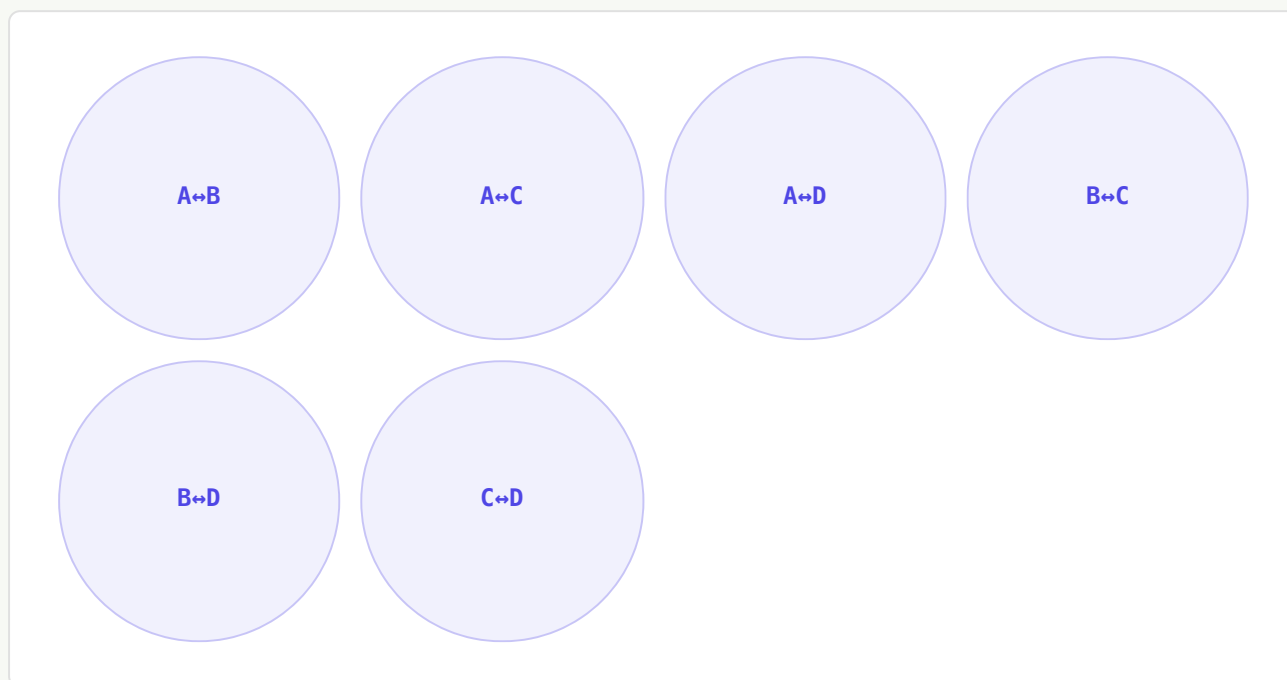


Figure 2 — a 4-person call: 6 direct connections $(n \cdot (n-1) / 2)$.

STUN resolves a direct path where the network allows it; a Metered TURN relay covers the networks where it can't. Screen share and camera video are both additive — they attach a track to the connection that already exists

rather than opening a separate call mode, so captions and translation never pause for either. Screen share is 1:1 only, since no iOS browser has ever shipped `getDisplayMedia`; camera video has no such gap and works everywhere `getUserMedia` does.

Data model and security

Postgres, via Supabase, with Row Level Security on every table — a user's own client can only ever see rows the database itself has decided belong to them, independent of what the application code does or forgets to check.

- **Secrets never reach the browser.** The client is handed a short-lived, scoped AssemblyAI token for streaming audio; the Anthropic key never leaves the server. Translation always routes browser → server → Claude → browser.
- **Media stays peer-to-peer.** As above — audio and video are never proxied or stored server-side; only the signalling exchange briefly touches Supabase Realtime.
- **Chat translations are persisted, call translations are not.** A chat thread's translations are computed once per message per language and cached. A call's translation is live and ephemeral — spoken and captioned, never recorded, except in the short post-call summary Claude writes afterward, one version per language actually present on the call.

Language coverage: naming a real constraint

OHUN supports seven languages for chat and voice notes — English, French, Spanish, German, Portuguese, Italian, and Yorùbá. Calls support six of those seven; Yorùbá is not yet available on a call.

English

Français

Español

Deutsch

Português

Italiano

Yorùbá — chat & voice notes only

The reason is a real, specific one, not a policy choice: AssemblyAI's realtime Universal-Streaming models, which calls depend on for low-latency transcription, have no Yorùbá model. Voice notes don't share that limit, because they use AssemblyAI's *batch* transcription API instead — a separate model with materially broader language coverage, Yorùbá included, at the cost of the latency that makes it unsuitable for a live call.

Rather than silently degrade — accept audio in an unsupported language and transcribe it badly, or worse, transcribe a different language with false confidence — OHUN surfaces this as a clear, specific state: an account set to Yorùbá can message and send voice notes normally, and sees a direct explanation, not a generic error, if it tries to place or join a call. Making Yorùbá real on calls means adding a realtime speech-to-text path for it specifically — a streaming-capable model or a different provider for that language — which is on the roadmap (§9) rather than simulated in the meantime.

Implementation: the call/room language gate lives in `lib/calls/actions.ts` and `lib/rooms/actions.ts`; voice-note transcription, including the batch-API path and its confidence handling, lives in `lib/chat/transcribe.ts`.

Group calls and scale

Group calls run up to 7 participants. A full mesh's connection count grows with the square of room size — $n \cdot (n-1)/2$ connections, each participant transmitting audio (and any camera) to every other participant directly. That's the right trade for a small room: no media server to operate, genuinely peer-to-peer, and low latency. It is not the right trade past a handful of participants, where per-connection cost compounds fast enough that a selective forwarding unit — a media server that receives once from each participant and forwards selectively — becomes the correct architecture instead.

Reliability notes

Two details worth naming specifically, because they're the kind of failure mode that looks fine in a demo and breaks quietly in the field:

Confidence, not just detection

Automatic language detection on a transcription request doesn't fail cleanly on a low-confidence guess by default — it can return a confident-looking transcript for the wrong language entirely. OHUN sets an explicit confidence threshold on detected transcriptions and treats anything below it as a failure, then retries with the account's own declared language stated explicitly rather than guessed. The alternative — trusting a silent low-confidence guess — produces exactly the failure mode it looks like: text that appears to transcribe correctly and then translates into nonsense, with no visible error anywhere in the chain.

Negotiation races are the common case, not the edge case

In a live call, a local renegotiation (muting, adding a camera) and an incoming offer from the other side are not a rare coincidence — with two people actively using the call, they happen constantly. The perfect-negotiation pattern in §4 exists because treating that as an edge case produces intermittent, hard-to-reproduce dropped connections instead.

Roadmap

1. **Yorùbá on calls.** A realtime speech-to-text path for Yorùbá — closing the one remaining gap between what chat supports and what calls support.
2. **Additional languages.** Grown deliberately, one at a time, verified against real transcription and translation quality before being offered rather than listed speculatively.
3. **Group calls past 7.** A selective forwarding unit to replace the mesh topology once room size outgrows what direct peer connections can carry well.

Conclusion

The interesting engineering in OHUN is less about any single model and more about keeping the conversation's shape intact while three separate systems — speech-to-text, translation, and delivery — do their work underneath it in real time, and about being exact instead of optimistic when one of those systems has a real boundary. Yorùbá on chat but not yet on calls is not a rough edge to paper over; it is what an honest system looks like while a genuine constraint is still being solved.

APPENDIX — STACK

Framework	Next.js 16 (App Router, Server Actions, Turbopack), React 19
Data	Supabase — Postgres with Row Level Security, Auth, Realtime, Storage
Speech-to-text	AssemblyAI — Universal-Streaming (realtime) and batch transcription
Translation & summaries	Anthropic Claude
Calling	WebRTC (RTCPeerConnection directly), STUN + Metered TURN
Notifications	Web Push (VAPID), installable PWA including iOS Safari
Styling	Tailwind CSS 4

OHUN/WP-1 – Real-time translation, built like a conversation

github.com/TeevincsCrypt/ohun